One wrong communication choice can slow down your entire system!

What are Transport Layers in Microservices?

- Define how services communicate with each other
- Handle data transfer between services
- Can be synchronous or asynchronous
- Core part of system performance and scalability

TCP (Default & Simple)

- Basic request-response communication
- Built-in transport in NestJS
- Fast and lightweight
- Best for simple internal communication
- No advanced features (no streaming, no pub/sub)

Redis (Pub/Sub Messaging)

- Asynchronous communication using events
- Uses publish/subscribe pattern
- Great for real-time systems (notifications, updates)
- Decouples services (no direct dependency)
- Not ideal for complex data pipelines

gRPC (High Performance)

- Uses Protocol Buffers (binary format)
- Extremely fast and efficient
- Supports streaming (real-time data flow)
- Strong contract-based communication
- Best for high-performance systems

TCP vs Redis vs gRPC

- TCP → Simple & direct communication
- Redis → Event-driven & decoupled
- gRPC → High-performance & structured

When to Use What?

- TCP → Small projects, simple services
- Redis → Event-based systems, notifications
- gRPC → High-scale, performance-critical systems

Common Mistakes

- Using TCP for event-driven systems
- Using Redis for request-response
- Ignoring performance needs
- Not choosing based on use case